

the Extended Kalman Filter (EKF) subscribing to the IMU to calculate exact vehicle pose.

2.1.2 Part 2: Implementation - The Publisher

We will build a `SpeedPublisher` that simulates reading a wheel encoder and broadcasting the vehicle's speed at 10Hz. We will use a standard ROS 2 message type: `std_msgs/msg/Float32`.

Modern C++ 14/17 Implementation

File: `src/speed_publisher.cpp`

```
1  #include <chrono>
2  #include <memory>
3  // Core ROS 2 C++ API
4  #include "rclcpp/rclcpp.hpp"
5  // The specific message type we are broadcasting
6  #include "std_msgs/msg/float32.hpp"
7
8  using namespace std::chrono_literals;
9
10 class SpeedPublisher : public rclcpp::Node
11 {
12 public:
13     SpeedPublisher() : Node("speed_sensor_node"), current_speed_(0.0)
14     {
15         // 1. Create the Publisher.
16         // Syntax: create_publisher<MessageType>("topic_name",
17         //                                     qos_history_depth)
18         publisher_ = this->create_publisher<std_msgs::msg::
19             Float32>("/vehicle/speed", 10);
20
21         // 2. Create a timer to fire at 10Hz (100ms) to trigger
22         // the publish loop
23         timer_ = this->create_wall_timer(
24             100ms, std::bind(&SpeedPublisher::timer_callback,
25                             this));
26
27         RCLCPP_INFO(this->get_logger(), "Speed Publisher
28             initialized on /vehicle/speed");
29     }
30
31 private:
32     void timer_callback()
33     {
34         // 3. Instantiate the message object
35         auto message = std_msgs::msg::Float32();
36         message.data = current_speed_;
37
38         // 4. Publish the message to the DDS network
```

```
34     publisher_ -> publish(message);
35
36     // Simulate slight acceleration for our test
37     current_speed_ += 0.5;
38 }
39
40 // Class members
41 rclcpp::TimerBase::SharedPtr timer_;
42 rclcpp::Publisher<std_msgs::msg::Float32>::SharedPtr
43     publisher_;
44 float current_speed_;
45 };
46
47 int main(int argc, char * argv[])
48 {
49     rclcpp::init(argc, argv);
50     rclcpp::spin(std::make_shared<SpeedPublisher>());
51     rclcpp::shutdown();
52     return 0;
53 }
```

Python Implementation

File: speed_publisher.py

```
1  import rclpy
2  from rclpy.node import Node
3  # The specific message type we are broadcasting
4  from std_msgs.msg import Float32
5
6  class SpeedPublisher(Node):
7      def __init__(self):
8          super().__init__('speed_sensor_node')
9          self.current_speed_ = 0.0
10
11      # 1. Create the Publisher.
12      # Syntax: create_publisher(MessageType, 'topic_name',
13          qos_history_depth)
14      self.publisher_ = self.create_publisher(Float32, '/
15          vehicle/speed', 10)
16
17      # 2. Create a timer to fire at 10Hz (0.1 seconds)
18      self.timer = self.create_timer(0.1, self.timer_callback)
19      self.get_logger().info('Speed Publisher initialized on /
20          vehicle/speed')
21
22      def timer_callback(self):
23          # 3. Instantiate the message object
24          msg = Float32()
25          msg.data = self.current_speed_
26
27          # 4. Publish the message
28          self.publisher_.publish(msg)
```

```
25         self.publisher_.publish(msg)
26
27         self.current_speed_ += 0.5
28
29     def main(args=None):
30         rclpy.init(args=args)
31         node = SpeedPublisher()
32         rclpy.spin(node)
33         node.destroy_node()
34         rclpy.shutdown()
35
36     if __name__ == '__main__':
37         main()
```

2.1.3 Part 3: Implementation - The Subscriber

Now we build a `DashboardSubscriber` that actively listens to `/vehicle/speed` and reacts every time a new packet arrives.

Modern C++ 14/17 Implementation

File: `src/dashboard_subscriber.cpp`

```
1  #include <memory>
2  #include "rclcpp/rclcpp.hpp"
3  #include "std_msgs/msg/float32.hpp"
4
5  // Required for the std::bind placeholder _1
6  using std::placeholders::_1;
7
8  class DashboardSubscriber : public rclcpp::Node
9  {
10 public:
11     DashboardSubscriber() : Node("dashboard_node")
12     {
13         // 1. Create the Subscriber.
14         // Syntax: create_subscription<MessageType>("topic_name",
15             //      qos_depth, callback_function)
16         subscription_ = this->create_subscription<std_msgs::msg::
17             Float32>(
18             "/vehicle/speed", 10, std::bind(&DashboardSubscriber
19                 ::speed_callback, this, _1));
20
21         RCLCPP_INFO(this->get_logger(), "Dashboard Subscriber
22             listening to /vehicle/speed");
23     }
24
25 private:
26     // 2. The Callback Function. Triggered automatically by the
27     //      Executor when data arrives.
28     // It takes a Shared Pointer to the incoming message to
29     //      prevent unnecessary memory copying.
```

```
24     void speed_callback(const std_msgs::msg::Float32::SharedPtr
25                          msg) const
26     {
27         RCLCPP_INFO(this->get_logger(), "Dashboard reading: %.2f
28                     m/s", msg->data);
29     }
30
31     rclcpp::Subscription<std_msgs::msg::Float32>::SharedPtr
32     subscription_;
33 };
34
35 int main(int argc, char * argv[])
36 {
37     rclcpp::init(argc, argv);
38     rclcpp::spin(std::make_shared<DashboardSubscriber>());
39     rclcpp::shutdown();
40     return 0;
41 }
```

Python Implementation

File: dashboard_subscriber.py

```
1  import rclpy
2  from rclpy.node import Node
3  from std_msgs.msg import Float32
4
5  class DashboardSubscriber(Node):
6      def __init__(self):
7          super().__init__('dashboard_node')
8
9          # 1. Create the Subscriber.
10         self.subscription = self.create_subscription(
11             Float32,
12             '/vehicle/speed',
13             self.speed_callback,
14             10)
15         self.subscription # Prevent unused variable warning
16         self.get_logger().info('Dashboard Subscriber listening to
17                               /vehicle/speed')
18
19         # 2. The Callback Function. Note the 'msg' parameter.
20         def speed_callback(self, msg):
21             self.get_logger().info(f'Dashboard reading: {msg.data:.2f
22                                   } m/s')
23
24     def main(args=None):
25         rclpy.init(args=args)
26         node = DashboardSubscriber()
27         rclpy.spin(node)
28         node.destroy_node()
29         rclpy.shutdown()
```

```

28
29 if __name__ == '__main__':
30     main()

```

Best Practices & Common Pitfalls

- **Pitfall – The Silent Topic Mismatch:** If your C++ publisher publishes to `/vehicle/speed` but your Python subscriber is accidentally listening to `/Vehicle/Speed` (capitalized) or `/vehicle/velocity`, there will be zero compiler errors and zero runtime errors. The DDS network simply routes them as two completely independent paths. *Always use `ros2 topic list` and `ros2 topic echo <topic_name>` to verify data flow before doubting your code.*
- **Best Practice – Zero-Copy Pointers in C++:** Notice in the C++ subscriber, the callback argument is `const std_msgs::msg::Float32::SharedPtr msg`. We pass by a constant shared pointer. For a simple float, it doesn't matter. But if that message is a 15-megabyte PointCloud from a 3D LiDAR, passing by value would cause a massive memory copy, creating a catastrophic CPU spike. Always use pointers for message callbacks.
- **Pitfall – The Missing Placeholder in C++:** When binding the callback in `create_subscription`, engineers often forget the `std::placeholders::_1`. This tells C++ that the callback function expects exactly one argument (the incoming message). Without `_1`, the code will refuse to compile with a deeply confusing template error.

2.1.4 Part 4: Inspecting the Data Stream (CLI Tools)

The "Why"

Nodes are black boxes once compiled. If the vehicle's steering is erratic, you don't know if the perception node is calculating the wrong angles, or if the motor controller is just failing to execute them. You need to "tap the wire" and read the data mid-flight.

The "How" (Architecture Level)

Just like `ros2 node` commands, `ros2 topic` commands temporarily spin up an anonymous, hidden node in your terminal. This node subscribes to the requested topic, processes the incoming DDS packets (either printing them, timing them, or analyzing the metadata), and then cleanly shuts down when you press `Ctrl+C`.

1. The Typographical Check: `ros2 topic list -t`

The "Why": You need to know what streams are active on the network and, critically, what language (message type) they speak.

The "How": The `-t` flag (type) is mandatory for serious debugging. It forces the DDS discovery protocol to return not just the topic names, but the exact message interface associated with them.

Real-world Autonomous Racing Use Case: If your autonomous technical team is trying to integrate a new IMU driver, you run this to confirm whether the driver outputs standard `sensor_msgs/msg/Imu` data, or if the manufacturer used a proprietary custom

message type that will break your Extended Kalman Filter.

Implementation

Terminal

```
ros2 topic list -t
```

```
# Expected Terminal Output:
# /parameter_events [rcl_interfaces/msg/ParameterEvent]
# /rosout [rcl_interfaces/msg/Log]
# /vehicle/speed [std_msgs/msg/Float32]
```

2. The Wiretap: `ros2 topic echo <topic_name>`

The "Why": You need to see the actual payload. Are we publishing zeros? Are we publishing NaN because a matrix inversion failed in our C++ code?

The "How": This command forces your terminal to subscribe to the topic and print the deserialized data to standard output.

Real-world Autonomous Racing Use Case: The vehicle is refusing to accelerate. You echo the `/cmd_vel` (Command Velocity) topic. If the terminal prints `linear: {x: 0.0, y: 0.0, z: 0.0}`, you instantly know the problem is upstream in the planner, not a physical hardware failure in the motor.

Implementation

Terminal

```
ros2 topic echo /vehicle/speed
```

```
# Expected Terminal Output:
# data: 0.5
# ---
# data: 1.0
# ---
# data: 1.5
```

3. The Heartbeat Monitor: `ros2 topic hz <topic_name>`

The "Why": This is arguably the most important command for sensor integration. Algorithms are designed to expect data at specific frequencies. If the data is too slow, the system lags.

The "How": This command subscribes to the topic, records the timestamp of each incoming message, and calculates the moving average frequency (Hertz), min/max delay, and standard deviation.

Real-world Autonomous Racing Use Case: You configure a 3D LiDAR to spin at 20Hz. However, your team notices the obstacle avoidance is reacting sluggishly. You run `ros2 topic hz /scan`. The terminal shows an average rate of 4.2 Hz. You immediately know your network bandwidth is choked or the LiDAR driver is fundamentally misconfigured, dropping 75% of the frames.

Implementation

Terminal

```
ros2 topic hz /vehicle/speed
```

```
# Expected Terminal Output:
```

```
# average rate: 9.998
```

```
# min: 0.100s max: 0.100s std dev: 0.00014s window: 10
```

4. The Graph Inspector: `ros2 topic info <topic_name> --verbose`

The "Why": You have verified a topic exists, but the subscriber node is still not receiving data. You need to see the "routing table" for that specific topic.

The "How": This command queries the ROS 2 graph to list exactly how many publishers and subscribers are attached to a topic. By appending `--verbose`, you force it to reveal the DDS Quality of Service (QoS) profiles for every connected node.

Real-world Autonomous Racing Use Case: You are trying to stream a high-resolution camera to a dashboard UI. The camera node is running, the UI node is running, but the screen is black. You run `ros2 topic info /camera/image_raw --verbose` and discover the Camera Publisher is set to Best Effort QoS, but the UI Subscriber is strictly demanding Reliable QoS. The middleware is silently blocking the connection.

Implementation

Terminal

```
ros2 topic info /vehicle/speed --verbose
```

```
# Expected Terminal Output:
```

```
# Type: std_msgs/msg/Float32
```

```
# Publisher count: 1
```

```
# Node name: speed_sensor_node
```

```
# Node namespace: /
```

```
# Topic type: std_msgs/msg/Float32
```

```
# Endpoint type: PUBLISHER
```

```
# GID: 01.0f.12...
```

```
# QoS profile:
```

```
# Reliability: RELIABLE
```

```
# History (Depth): KEEP_LAST (10)
```

```
# Durability: VOLATILE
```

```
# Subscriber count: 1
```

```
# ...
```

Recommended Video Tutorial:

[How to Write C++ Subscriber and Publisher Nodes in ROS2 Humble](#)

from Scratch

This tutorial provides an excellent visual walkthrough of compiling and testing these exact C++ subscriber and publisher concepts in a live terminal environment.

2.2 Module 2.2: Services (Synchronous Requests)

2.2.1 Part 1: The "Why" & The "How" (Client/Server Architecture)

The "Why"

Topics use a "fire-and-forget" broadcast model. If you publish a message to enable the emergency brakes on a Topic, the Publisher has no idea if the motor controller actually received it or acted on it.

For critical, one-off commands, you need a Request/Response architecture. You need a guarantee. A Service allows one node (the **Client**) to send a specific Request to another node (the **Server**) and pause its own high-level logic until it receives a concrete Response (Success/Failure).

The "How" (Architecture Level)

Under the hood in the DDS layer, a ROS 2 Service is actually constructed using two hidden, paired DDS Topics.

- The Client publishes to a hidden **Request** topic and attaches a unique Sequence ID to the message.
- The Server processes the request and publishes to a hidden **Reply** topic, attaching that exact same Sequence ID.
- The Client matches the ID, accepts the response, and resumes its logic.

Unlike standard Topics, Services are 1-to-1 (or Many-to-1). Only one node can act as the Service Server, though multiple clients can send it requests.

Real-world Autonomous Racing Use Cases

- **Resetting the Extended Kalman Filter (EKF):** If your car spins out and the `robot_localization` node loses its pose estimation, you do not stream "reset" at 10Hz. You call an `/ekf/reset` service once, and the EKF replies "Reset Complete" so the planner knows it can safely resume navigation.
- **Sensor Calibration Routine:** Triggering the steering servo to sweep from lock-to-lock to find its center before the race begins.
- **State Management:** Changing the state of the PID controller (e.g., toggling it from Active to Inactive) using a boolean flag.

2.2.2 Part 2: Implementation - The Service Server

We will build a `PID_State_Manager` node. It will host a service that allows other nodes to turn the autonomous steering PID controller on or off. We will use the standard `std_srvs/srv/SetBool` interface.

Note: A `SetBool` request contains a boolean `data`. The response contains a boolean `success` and a string `message`.

Modern C++ 14/17 Implementation

File: `src/pid_server.cpp`

```

1  #include <memory>
2  #include "rclcpp/rclcpp.hpp"
3  // The specific service interface type
4  #include "std_srvs/srv/set_bool.hpp"
5
6  using std::placeholders::_1;
7  using std::placeholders::_2;
8
9  class PIDServer : public rclcpp::Node
10 {
11 public:
12     PIDServer() : Node("pid_manager_node"), pid_active_(false)
13     {
14         // 1. Create the Service Server.
15         // Syntax: create_service<ServiceType>("service_name",
16             // callback_function)
17         service_ = this->create_service<std_srvs::srv::SetBool>(
18             "/control/toggle_pid",
19             std::bind(&PIDServer::handle_toggle_request, this, _1
20                 , _2));
21
22         RCLCPP_INFO(this->get_logger(), "PID Manager Service
23             Ready on /control/toggle_pid");
24     }
25
26 private:
27     // 2. The Service Callback.
28     // It takes a Shared Pointer to the Request, and a Shared
29     // Pointer to the Response.
30     void handle_toggle_request(
31         const std::shared_ptr<std_srvs::srv::SetBool::Request>
32             request,
33         std::shared_ptr<std_srvs::srv::SetBool::Response>
34             response)
35     {
36         // Act on the request
37         pid_active_ = request->data;
38
39         if (pid_active_) {

```

```
34         RCLCPP_INFO(this->get_logger(), "Engaging PID
           Controller...");
35         response->message = "PID Successfully Engaged.";
36     } else {
37         RCLCPP_WARN(this->get_logger(), "Disengaging PID
           Controller!");
38         response->message = "PID Successfully Disengaged.";
39     }
40
41     // Populate the response
42     response->success = true;
43
44     // The callback finishes, and the Executor automatically
       sends the 'response' back to the Client.
45 }
46
47 rclcpp::Service<std_srvs::srv::SetBool>::SharedPtr service_;
48 bool pid_active_;
49 };
50
51 int main(int argc, char **argv) {
52     rclcpp::init(argc, argv);
53     rclcpp::spin(std::make_shared<PIDServer>());
54     rclcpp::shutdown();
55     return 0;
56 }
```

Python Implementation

File: *pid_server.py*

```
1  import rclpy
2  from rclpy.node import Node
3  from std_srvs.srv import SetBool
4
5  class PIDServer(Node):
6      def __init__(self):
7          super().__init__('pid_manager_node')
8          self.pid_active_ = False
9
10         # 1. Create the Service Server
11         self.srv = self.create_service(
12             SetBool,
13             '/control/toggle_pid',
14             self.handle_toggle_request)
15
16         self.get_logger().info('PID Manager Service Ready on /
           control/toggle_pid')
17
18         # 2. The Service Callback
19         def handle_toggle_request(self, request, response):
20             self.pid_active_ = request.data
```

```
21
22     if self.pid_active_:
23         self.get_logger().info('Engaging PID Controller...')
24         response.message = 'PID Successfully Engaged.'
25     else:
26         self.get_logger().warning('Disengaging PID Controller
27                                     !')
28         response.message = 'PID Successfully Disengaged.'
29
30     response.success = True
31
32     # In Python, you MUST return the response object
33     return response
34
35 def main(args=None):
36     rclpy.init(args=args)
37     node = PIDServer()
38     rclpy.spin(node)
39     node.destroy_node()
40     rclpy.shutdown()
41
42 if __name__ == '__main__':
43     main()
```

2.2.3 Part 3: Implementation - The Service Client (With Async Best Practices)

Now, we build a `Safety_Dashboard_Client`. It will request the PID to turn off.

Modern C++ 14/17 Implementation

File: `src/safety_client.cpp`

```
1  #include <chrono>
2  #include <memory>
3  #include "rclcpp/rclcpp.hpp"
4  #include "std_srvs/srv/set_bool.hpp"
5
6  using namespace std::chrono_literals;
7
8  class SafetyClient : public rclcpp::Node
9  {
10 public:
11     SafetyClient() : Node("safety_dashboard_node")
12     {
13         // 1. Create the Client
14         client_ = this->create_client<std_srvs::srv::SetBool>("/
15             control/toggle_pid");
16
17         // 2. Wait for the server to actually exist on the
18             network before sending requests
```

```
17     while (!client_->wait_for_service(1s)) {
18         if (!rclcpp::ok()) {
19             RCLCPP_ERROR(this->get_logger(), "Interrupted
20                 while waiting for the service. Exiting.");
21             return;
22         }
23         RCLCPP_INFO(this->get_logger(), "Waiting for PID
24             Service to appear on network...");
25     }
26     send_disable_command();
27 }
28 private:
29 void send_disable_command()
30 {
31     // 3. Formulate the request
32     auto request = std::make_shared<std_srvs::srv::SetBool::
33         Request>();
34     request->data = false; // Turn PID OFF
35     RCLCPP_WARN(this->get_logger(), "Sending EMERGENCY
36         DISABLE command to PID...");
37     // 4. CRITICAL: Send the request Asynchronously.
38     // We bind a callback function that will execute ONLY
39     when the response arrives.
40     client_->async_send_request(
41         request,
42         std::bind(&SafetyClient::response_callback, this, std
43             ::placeholders::_1));
44     // 5. The Response Callback
45     void response_callback(rclcpp::Client<std_srvs::srv::SetBool
46         >::SharedFuture future)
47     {
48         // Extract the response from the future object
49         auto response = future.get();
50         if (response->success) {
51             RCLCPP_INFO(this->get_logger(), "Server replied: %s",
52                 response->message.c_str());
53         } else {
54             RCLCPP_ERROR(this->get_logger(), "Failed to disable
55                 PID!");
56         }
57     }
58     rclcpp::Client<std_srvs::srv::SetBool>::SharedPtr client_;
```

```
59 int main(int argc, char **argv) {
60     rclcpp::init(argc, argv);
61     rclcpp::spin(std::make_shared<SafetyClient>());
62     rclcpp::shutdown();
63     return 0;
64 }
```

Python Implementation

File: safety_client.py

```
1  import rclpy
2  from rclpy.node import Node
3  from std_srvs.srv import SetBool
4
5  class SafetyClient(Node):
6      def __init__(self):
7          super().__init__('safety_dashboard_node')
8
9          # 1. Create the Client
10         self.client = self.create_client(SetBool, '/control/
            toggle_pid')
11
12         # 2. Wait for the server to exist
13         while not self.client.wait_for_service(timeout_sec=1.0):
14             self.get_logger().info('Waiting for PID Service to
                appear on network...')
15
16         self.send_disable_command()
17
18         def send_disable_command(self):
19             # 3. Formulate the request
20             request = SetBool.Request()
21             request.data = False # Turn PID OFF
22
23             self.get_logger().warning('Sending EMERGENCY DISABLE
                command to PID...')
24
25             # 4. CRITICAL: Send the request Asynchronously.
26             future = self.client.call_async(request)
27             # Attach a callback to the future object
28             future.add_done_callback(self.response_callback)
29
30         # 5. The Response Callback
31         def response_callback(self, future):
32             try:
33                 response = future.result()
34                 if response.success:
35                     self.get_logger().info(f'Server replied: {
                        response.message}')
36                 else:
37                     self.get_logger().error('Failed to disable PID!')
```

```
38         except Exception as e:
39             self.get_logger().error(f'Service call failed: {e}')
40
41     def main(args=None):
42         rclpy.init(args=args)
43         node = SafetyClient()
44         rclpy.spin(node)
45         node.destroy_node()
46         rclpy.shutdown()
47
48     if __name__ == '__main__':
49         main()
```

2.2.4 Part 4: Best Practices & Pitfalls (CRITICAL: The Dead-lock Trap)

This is the most common reason senior engineers have to rescue junior engineers' code.

The Deadly Pitfall: Blocking the Executor (Deadlocks)

In ROS 1, it was common to make a synchronous service call (`client->send_request()` in C++ or `client.call()` in Python) directly inside a timer callback. In ROS 2, if you do this while using the default Single-Threaded Executor, your entire node will permanently freeze in a deadlock.

Why does this happen?

1. Your timer fires. The single Executor thread enters the timer callback.
2. Inside the callback, you send a synchronous request to the Server and wait. You are telling the thread, "Do not move until the response arrives."
3. The Server processes the request and sends the response back over the DDS network.
4. The response arrives at your Client node, waiting in the queue.

The Trap: The only thread capable of reading that response queue is currently frozen inside your timer callback, waiting for the response! The system is permanently deadlocked.

How Professionals Handle This:

- **Always use Asynchronous calls (`async_send_request` / `call_async`):** As demonstrated in the code above, you fire the request and immediately exit the function. You provide a callback that the Executor will run only when the response naturally arrives in the queue.
- **Callback Groups & Multi-Threaded Executors:** If you are writing a complex state machine (like an autonomous mission planner) where you absolutely *must* block and wait sequentially, you must place the Service Client into a separate `MutuallyExclusiveCallbackGroup` and spin your node using an `rclcpp::executors::MultiThreadedExecutor`. (We will cover this deeply in Phase 5: Advanced Node Architecture).

2.3 Module 2.3: Actions (Asynchronous, Long-Running Tasks)

2.3.1 Part 1: The "Why" & The "How" (Proactive Cancellation)

The "Why"

Your high-level planner (the Client) is the brain. The Action Server is the muscle. If the brain's perception nodes detect a sudden obstacle at 25 meters, the brain must instantly instruct the muscle to abort the 50-meter drive command.

The "How" (Architecture Level)

When an Action Client sends a goal, it receives a Goal Handle. This handle is your direct line of communication to that specific executing task. To cancel, the Client takes that Goal Handle and sends an asynchronous request to the Server's hidden Cancel Service. The Server processes it, safely winds down its hardware loop (as we coded in the previous module), and returns a `CANCELED` result to the Client.

Real-world Autonomous Racing Use Cases

- **Dynamic Obstacle Avoidance:** Aborting a path-following action because a slower car swerved into your lane.
- **Emergency Pit Stop:** Canceling an active track lap action to immediately route the vehicle back to the pit lane due to low battery voltage.

2.3.2 Part 2: Implementation - The Action Client (With Safety Preemption)

We will build the `SafetyPlannerClient`. It will request a 50-meter drive. When the feedback indicates we have reached 25 meters, it will simulate detecting an obstacle and trigger the cancellation.

Modern C++ 14/17 Implementation

File: `src/safety_planner_client.cpp`

```

1  #include <memory>
2  #include "rclcpp/rclcpp.hpp"
3  #include "rclcpp_action/rclcpp_action.hpp"
4  #include "racing_interfaces/action/drive_distance.hpp"
5
6  using namespace std::placeholders;
7
8  class SafetyPlannerClient : public rclcpp::Node
9  {
10 public:
11     using DriveDistance = racing_interfaces::action::
        DriveDistance;

```

```
12     using GoalHandleDrive = rclcpp_action::ClientGoalHandle<
13         DriveDistance>;
14
15     SafetyPlannerClient() : Node("safety_planner_client")
16     {
17         // 1. Create the Action Client
18         this->client_ptr_ = rclcpp_action::create_client<
19             DriveDistance>(
20                 this, "/control/drive_distance");
21
22         // Wait for server to appear on the network
23         while (!this->client_ptr_->wait_for_action_server(std::
24             chrono::seconds(1))) {
25             RCLCPP_INFO(this->get_logger(), "Waiting for Drive
26                 Action Server...");
27         }
28
29         // Send the command immediately for this example
30         this->send_goal();
31     }
32
33 private:
34     rclcpp_action::Client<DriveDistance>::SharedPtr client_ptr_;
35     bool cancel_triggered_ = false; // Guard flag to ensure we
36         only send cancel once
37
38     void send_goal()
39     {
40         RCLCPP_INFO(this->get_logger(), "Planner requesting 50-
41             meter drive.");
42
43         auto goal_msg = DriveDistance::Goal();
44         goal_msg.target_distance = 50.0;
45
46         auto send_goal_options = rclcpp_action::Client<
47             DriveDistance>::SendGoalOptions();
48
49         // Bind our 3 crucial async callbacks
50         send_goal_options.goal_response_callback =
51             std::bind(&SafetyPlannerClient::
52                 goal_response_callback, this, _1);
53         send_goal_options.feedback_callback =
54             std::bind(&SafetyPlannerClient::feedback_callback,
55                 this, _1, _2);
56         send_goal_options.result_callback =
57             std::bind(&SafetyPlannerClient::result_callback, this
58                 , _1);
59
60         // 2. Send the Goal Asynchronously
61         this->client_ptr_->async_send_goal(goal_msg,
62             send_goal_options);
```



```

52     }
53
54     // Callback 1: Did the server accept our request?
55     void goal_response_callback(const GoalHandleDrive::SharedPtr
56         & goal_handle)
57     {
58         if (!goal_handle) {
59             RCLCPP_ERROR(this->get_logger(), "Goal was rejected
60                 by server");
61         } else {
62             RCLCPP_INFO(this->get_logger(), "Goal accepted by
63                 server, waiting for result");
64         }
65     }
66
67     // Callback 2: Continuous Feedback Stream & The Safety
68         Trigger
69     void feedback_callback(
70         GoalHandleDrive::SharedPtr goal_handle,
71         const std::shared_ptr<const DriveDistance::Feedback>
72             feedback)
73     {
74         RCLCPP_INFO(this->get_logger(), "Feedback: Driven %.2f
75             meters", feedback->distance_traveled);
76
77         // --- THE PREEMPTION TRIGGER ---
78         if (feedback->distance_traveled >= 25.0 && !this->
79             cancel_triggered_) {
80             RCLCPP_WARN(this->get_logger(), "OBSTACLE DETECTED!
81                 Sending Cancel Request...");
82
83             // Asynchronously send the cancel request
84             this->client_ptr->async_cancel_goal(goal_handle);
85             this->cancel_triggered_ = true; // Prevent spamming
86                 cancel requests
87         }
88     }
89
90     // Callback 3: The Final Outcome
91     void result_callback(const GoalHandleDrive::WrappedResult &
92         result)
93     {
94         switch (result.code) {
95             case rclcpp_action::ResultCode::SUCCEEDED:
96                 RCLCPP_INFO(this->get_logger(), "Goal succeeded!
97                     Time: %.2f", result.result->total_time_elapsed
98                     );
99                 break;
100             case rclcpp_action::ResultCode::ABORTED:
101                 RCLCPP_ERROR(this->get_logger(), "Goal was
102                     aborted by server.");

```

```
90         break;
91         case rclcpp_action::ResultCode::CANCELED:
92             // We expect this branch to trigger due to our
93             // preemption!
94             RCLCPP_WARN(this->get_logger(), "Goal
95             successfully canceled by planner intervention!
96             ");
97             break;
98         default:
99             RCLCPP_ERROR(this->get_logger(), "Unknown result
100             code");
101             break;
102     }
103     rclcpp::shutdown();
104 }
105 };
106
107 int main(int argc, char ** argv){
108     rclcpp::init(argc, argv);
109     rclcpp::spin(std::make_shared<SafetyPlannerClient>());
110     return 0;
111 }
```

Python Implementation

File: *safety_planner_client.py*

```
1  import rclpy
2  from rclpy.node import Node
3  from rclpy.action import ActionClient
4  from racing_interfaces.action import DriveDistance
5
6  class SafetyPlannerClient(Node):
7      def __init__(self):
8          super().__init__('safety_planner_client')
9
10         # 1. Create the Action Client
11         self._action_client = ActionClient(self, DriveDistance, '
12         /control/drive_distance')
13         self.current_goal_handle = None
14         self.cancel_triggered_ = False
15
16         self.send_goal()
17
18     def send_goal(self):
19         self.get_logger().info('Waiting for Drive Action Server
20         ...')
21         self._action_client.wait_for_server()
22
23         goal_msg = DriveDistance.Goal()
24         goal_msg.target_distance = 50.0
```

```

24         self.get_logger().info('Planner requesting 50-meter drive
25         .')
26
27         # 2. Send the Goal Asynchronously
28         self._send_goal_future = self._action_client.
29             send_goal_async(
30                 goal_msg, feedback_callback=self.feedback_callback)
31
32         # Attach callback for when the server accepts/rejects the
33         goal
34         self._send_goal_future.add_done_callback(self.
35             goal_response_callback)
36
37         # Callback 1: Goal Acceptance
38         def goal_response_callback(self, future):
39             goal_handle = future.result()
40             if not goal_handle.accepted:
41                 self.get_logger().error('Goal rejected by server.')
42                 return
43
44             self.get_logger().info('Goal accepted by server, waiting
45             for result.')
46             self.current_goal_handle = goal_handle # Store handle for
47             cancellation
48
49             # Attach callback for the final result
50             self._get_result_future = goal_handle.get_result_async()
51             self._get_result_future.add_done_callback(self.
52                 get_result_callback)
53
54         # Callback 2: Continuous Feedback Stream & The Safety Trigger
55         def feedback_callback(self, feedback_msg):
56             distance = feedback_msg.feedback.distance_traveled
57             self.get_logger().info(f'Feedback: Driven {distance:.2f}
58             meters')
59
60         # --- THE PREEMPTION TRIGGER ---
61         if distance >= 25.0 and not self.cancel_triggered_ and
62             self.current_goal_handle:
63             self.get_logger().warning('OBSTACLE DETECTED! Sending
64             Cancel Request...')
65
66             # Asynchronously send the cancel request
67             self.current_goal_handle.cancel_goal_async()
68             self.cancel_triggered_ = True # Prevent spamming
69
70         # Callback 3: The Final Outcome
71         def get_result_callback(self, future):
72             result_status = future.result().status
73             from action_msgs.msg import GoalStatus

```

```

65         if result_status == GoalStatus.STATUS_SUCCEEDED:
66             self.get_logger().info('Goal succeeded!')
67         elif result_status == GoalStatus.STATUS_CANCELED:
68             # We expect this branch to trigger due to our
69             # preemption!
70             self.get_logger().warning('Goal successfully canceled
71                                     by planner intervention!')
72         elif result_status == GoalStatus.STATUS_ABORTED:
73             self.get_logger().error('Goal aborted by server.')
74
75         rclpy.shutdown()
76
77     def main(args=None):
78         rclpy.init(args=args)
79         node = SafetyPlannerClient()
80         rclpy.spin(node)
81
82 if __name__ == '__main__':
83     main()

```

2.3.3 Part 3: Best Practices & Pitfalls

Critical Action Client Pitfalls

- **The "Spamming Cancel" Pitfall:** Notice the `cancel_triggered` boolean flag in both implementations. A common mistake is putting the cancel logic inside the feedback callback without a guard flag. Because feedback streams at high frequencies, the moment the vehicle crosses 25 meters, the Client will rapidly fire hundreds of `async_cancel_goal` requests over the DDS network before the Server has time to respond and shut down. This can completely choke the middleware. Always use a state flag.
- **Losing the Goal Handle (Python):** In C++, the `feedback_callback` conveniently provides the `goal_handle` as an argument. In Python, the `feedback_callback` only receives the message. You must store `self.current_goal_handle` during the `goal_response_callback` so you have a reference to it when you need to cancel later.
- **Handling Server Rejection:** Never assume the server will accept your goal. If the car is currently in an "Emergency Stop" state, the Server might reply with REJECT to your drive command. If your Client code immediately tries to ask for a result on a rejected goal, it will throw a fatal null pointer exception. Always check if the goal was accepted before proceeding.

2.3.4 Part 4: Inspecting the Action Graph (CLI Tools)

The "Why"

When your vehicle refuses to move, or it starts moving and won't stop, you need to know immediately: Is the Server dead? Is the Client sending garbage data? Did the Server reject the goal? The Action CLI tools act as a temporary diagnostic Client/Server to give you instant answers on the track.

The "How" (Architecture Level)

Because an Action is secretly composed of five different Topics and Services, looking at raw `ros2 topic list` output when an Action is running is a nightmare—you will just see a massive wall of hidden `/control/drive_distance/_action/status`, `/feedback`, and `/send_goal` topics.

When you use `ros2 action` commands, the CLI wraps these complex underlying DDS calls into a clean, human-readable terminal output. It handles the asynchronous futures and state tracking for you behind the scenes.

1. The Interface Verification: `ros2 action list -t`

The "Why": Before you can send a goal or debug a connection, you need to know exactly what Actions are exposed to the network and what `.action` interface they speak.

The "How": Just like with topics, the `-t` flag forces the middleware to return the interface type.

Real-world Autonomous Racing Use Case: You just launched the massive Nav2 (Navigation2) stack. Before you try to write a high-level racing logic script, you run this command to verify that Nav2 successfully brought up the `/navigate_to_pose` Action Server and is ready to accept waypoints.

Implementation

Terminal

```
ros2 action list -t
```

```
# Expected Terminal Output:
```

```
# /control/drive_distance [racing_interfaces/action/DriveDistance]
```

```
# /navigate_to_pose [nav2_msgs/action/NavigateToPose]
```

2. The Graph & State Inspector: `ros2 action info <action_name>`

The "Why": You sent a goal to drive 50 meters, but the car is just sitting there. You need to know if the Action Server is actually online and if it's currently processing any goals.

The "How": This command queries the ROS 2 graph specifically for Action clients and servers attached to that namespace, and tells you how many nodes are connected.

Real-world Autonomous Racing Use Case: If you run this command and see **Action Servers: 0**, you instantly know your `drive_action_server` crashed or was never launched. If you see **Action Servers: 1**, but the car isn't moving, you know the problem is inside your goal logic, not the network.

Implementation

Terminal

```
ros2 action info /control/drive_distance
```

```
# Expected Terminal Output:  
# Action: /control/drive_distance  
# Action clients: 1  
#   /safety_planner_client  
# Action servers: 1  
#   /drive_action_server
```

3. The Manual Trigger: `ros2 action send_goal` (with Feedback)

The "Why": This is your most powerful hardware testing tool. Before you write a complex 200-line C++ Action Client, you want to manually test the Action Server. Does the motor actually spin when it receives a 5-meter drive command?

The "How": This command turns your terminal into a temporary Action Client. You pass the Action Name, the Interface Type, and the Goal Data formatted as a YAML string. Crucially, adding the `-f` (or `--feedback`) flag tells the terminal to dynamically subscribe to the hidden Feedback topic and print the stream to your screen while the task executes.

Real-world Autonomous Racing Use Case: Your car is jacked up in the pit garage. You want to test the low-level PID motor controllers without running the entire autonomous AI stack. You send a manual goal of 5 meters from your laptop and watch the feedback stream to ensure the wheel encoders are accurately reporting the distance traveled.

Implementation

Terminal

```
# Syntax: ros2 action send_goal <action_name> <action_type>
"{yaml_goal_data}" -f

ros2 action send_goal /control/drive_distance
racing_interfaces/action/DriveDistance "{target_distance: 5.0}"
-f

# Expected Terminal Output:
# Waiting for an action server to become available...
# Sending goal:
#     target_distance: 5.0
#
# Goal accepted with ID: 1e4b3c...
#
# Feedback:
#     distance_traveled: 1.0
# Feedback:
#     distance_traveled: 2.0
# Feedback:
#     distance_traveled: 3.0
# Feedback:
#     distance_traveled: 4.0
#
# Result:
#     total_time_elapsed: 5.0
#
# Goal finished with status: SUCCEEDED
```

Tech Lead Note:

If you don't use the `-f` flag, the terminal will just sit completely blank after "Goal accepted" until the final Result arrives. Always use `-f` during testing so you know the system hasn't frozen.

CHAPTER 3

Phase 3: The Command Center (Orchestration Tools)

3.1 Module 3.1: Parameters (Dynamic Configuration)

3.1.1 Part 1: The "Why" & The "How" (Distributed Parameters)

The "Why"

Parameters are configuration values associated with a node. They allow you to write generic, reusable code. Instead of writing a "Front Left Motor Node" and a "Rear Right Motor Node," you write one "Motor Node" and pass it a parameter indicating its physical location and max speed limits at startup.

The "How" (Architecture Level - A Massive Shift from ROS 1)

In ROS 1, there was a global "Parameter Server" hosted by the `roscore` master. If a node needed a parameter, it asked the master. This was a massive bottleneck and a single point of failure.

In ROS 2, **there is no central Parameter Server**. Instead, *every single node is its own parameter server*. Under the hood, when you instantiate an `rclcpp::Node` or `rclpy.node.Node`, the ROS 2 middleware automatically attaches several hidden Services to it (e.g., `~/get_parameters`, `~/set_parameters`). When you change a parameter from the terminal, you are actually just making a synchronous Service call to that specific node.

Real-world Autonomous Racing Use Cases

- **PID Tuning:** Adjusting Proportional, Integral, and Derivative gains while the car is actively driving.
- **Hardware Configuration:** Setting the camera resolution (e.g., 720p vs 1080p) or LiDAR rotation frequency based on the specific track environment.
- **Debug Toggles:** Toggling a boolean `publish_debug_images` parameter. When